guidance-for-visual-search-on-aws

Public

Watch 8 Fork 0

About

This Guidance shows how to create a visual search capability for ecommerce websites, allowing users to upload product images to find visually similar items and improve product discovery.

aws.amazon.com/solutions/guidance/visual-search-on-aws/

Readme
MIT-0 license
Code of conduct
Contributing
Security policy
Activity Custom properties

3 stars 8 watching 0 forks

Report repository

ma 3 Branches 0 Tags Go to file T Go to file Add file Code

Adibuer-lab Merge pull request #5 from aws-solutions-library-samples/dependabot/n... 705eab5 · last year

.github	enable workflows, codeowners	2 years ago
assets	initial commit	2 years ago
bin	add description to stack	2 years ago
deployment	initial commit	2 years ago
lib	updates to cdk	2 years ago
source	initial commit	2 years ago
test	initial commit	2 years ago
.gitignore	update deployment steps	2 years ago
.npmignore	initial commit	2 years ago
CODEOWNERS	enable workflows, codeowners	2 years ago
CODE_OF_CONDUCT.md	initial commit	2 years ago
CONTRIBUTING.md	initial commit	2 years ago
LICENSE	initial commit	2 years ago
README.md	Update README.md	2 years ago
cdk.context.json	initial commit	2 years ago
cdk.json	initial commit	2 years ago
jest.config.js	initial commit	2 years ago
package-lock.json	Bump aws-cdk-lib from 2.136.0 to 2.189.0	last year

package.json	Bump aws-cdk-lib from 2.136.0 to 2.189.0	last year
requirements.txt	update deployment steps	2 years ago

README Code of conduct Contributing MIT-0 license Security

Guidance for Simple Visual Search on AWS

Table of Contents

1. Overview
   - Solution Overview
   - Architecture Diagram
   - Cost
2. Prerequisites
3. Deployment Steps
4. Deployment Validation
5. Running the Guidance
6. Next Steps
7. Cleanup

Overview

With mobile commerce leading retail growth, seamless in-app visual search unlocks frictionless purchase experiences. Visual search has evolved from a novelty to a business necessity. While technically complex to build at scale, visual search drives measurable metrics around engagement, conversion, and revenue when implemented successfully. As consumer expectations and behaviors shift towards more visual and intuitive shopping, brands need robust visual search to deliver next-generation shopping experiences. With visual search powering shopping across channels, brands can provide consumers with flexibility and convenience while capturing valuable data on emerging visual trends and consumer preferences.

Visual search allows consumers to take or upload an image to search for visually similar images and products. This enables more intuitive and seamless product discoveries, allowing consumers to find products they see around them or even user-generated image content that resembles their particular style and tastes.

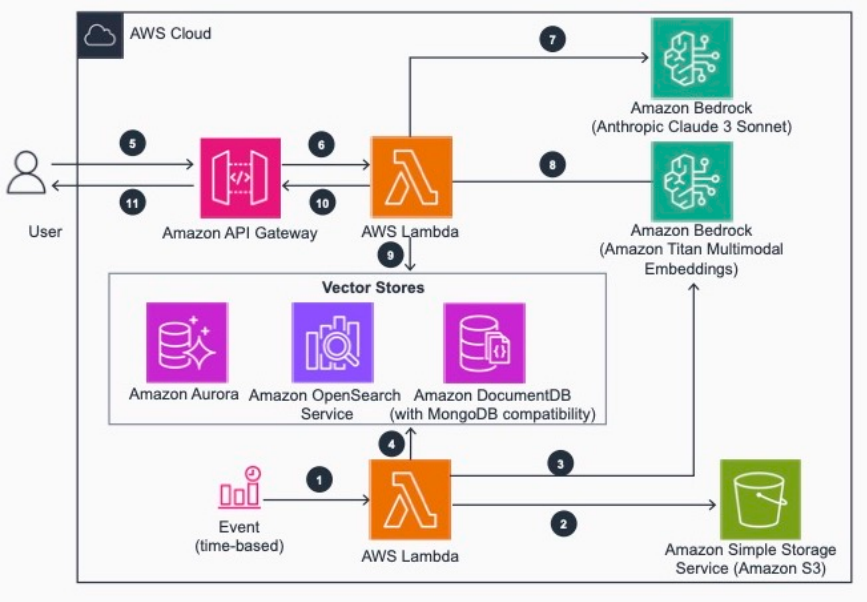
Developing accurate and scalable visual search is a complex technical challenge, which demands considerable investments in technology infrastructure and data management. However, recent advancements in generative AI and multimodal models are enabling exciting new possibilities in visual search.

This repo contains code that creates a visual search solution using services like Amazon Bedrock, Amazon Opensearch Serverless, Amazon Lambda etc.

Solution Overview

The solution is an implementation of semantic search, based on product images. To enable search by product images, we first need to create a vector store index of multimodal embeddings from the image and description of all products in the catalog. When you search with an image, the image is run through Claude Sonnet V3 to generate a caption for the image, and then both the input image and generated caption are used to create a multimodal embedding. This multimodal embedding is used to query the vector store index, which then returns the requested number of semantic search results based on similarity scores.

Architecture Diagram



1. A time-based Amazon EventBridge scheduler invokes an AWS Lambda Function to populate search index with multimodal embeddings and product meta-data.
2. The AWS Lambda Function first retrieves product feed stored as a JSON file in Amazon Simple Storage Service (Amazon S3).
3. The Lambda Function then invokes Amazon Bedrock's Titan Multimodal Embedding model to create vector embeddings for each product in the catalog, based on the primary image and description of the products.
4. The Lambda Function finally persists these vector embeddings as a k-NN vectors, along with product meta-data in the vector store (e.g. Amazon OpenSearch or Amazon DocumentDB or Amazon Aurora, etc.). This index is used as the source for semantic image search
5. The user initiates a visual search request through frontend application by uploading a product image.
6. The application uses Amazon API Gateway REST API to invoke a pre-configured proxy Lambda function to process the visual search request.
7. Lambda function first generates the caption for the input image using the Anthropic Claude 3 Sonnet model hosted on Amazon Bedrock. Optional step to create multimodal embedding based on both the input image and caption of the image, for improved search results.
8. Lambda function then invokes Amazon Titan Multimodal Embeddings model hosted on Amazon Bedrock to generate the multimodal embedding based on the input image uploaded by user and the image caption (if generated in step 7).
9. Lambda function then, performs a k-NN search on the vector store index, to find semantically similar results for the embedding generated in step 8.
10. The resultant semantic search results from the vector store are then filtered to eliminate any duplicates, enriched with product meta-data from the search index and passed back to API Gateway.
11. Finally, API Gateway response is returned to the client, to display the search results.

Cost

You are responsible for the cost of the AWS services used while running this Guidance. As of June 2024, the cost for running this Guidance with the default settings in the US East (N. Virginia) AWS Region is approximately $412.43 per month for processing 100,000 image searches.

We recommend creating a Budget through AWS Cost Explorer to help manage costs. Prices are subject to change. For full details, refer to the pricing webpage for each AWS service used in this Guidance.

Sample Cost Table

The following table provides a sample cost breakdown for deploying this Guidance with the default parameters in the US East (N.

Virginia) Region for one month.

AWS service	Dimensions	Cost [USD]
Amazon API Gateway	100,000 REST API calls per month	$ 0.35month
AWS Lambda	100,000 invocations per month	$ 0.68
Amazon Bedrock Titan Multimodal Embeddings feature	100,000 input images with corresponding text description	$ 166.00
Amazon Bedrock Anthropic Claude v3 Sonnet	100,000 input images per month	$ 4677.72
Amazon Opensearch Serverless	2 OCU(Indexing, Search and query cost) and 1GB storage	$ 350.42

Prerequisites

Operating System

These deployment instructions are optimized to best work on a pre-configured Amazon Linux 2023 AWS Cloud9 development environment. Refer to the Individual user setup for AWS Cloud9 for more information on how to set up Cloud9 as a user in the AWS account. Deployment using another OS may require additional steps, and configured python libraries (see Third-party tools).

Third-party tools

Before deploying the guidance code, ensure that the following required tools have been installed:

- AWS Cloud Development Kit (CDK) >= 2.126.0
- Python >= 3.8

AWS account requirements

1. Bedrock Model access for Claude 3 Sonnet and Amazon Titan Multimodal embeddings

aws cdk bootstrap

This Guidance uses AWS CDK. If you are using aws-cdk for the first time, please see the Bootstrapping section of the AWS Cloud Development Kit (AWS CDK) v2 developer guide, to provision the required resources, before you can deploy AWS CDK apps into an AWS environment.

Deployment Steps

1. In the Cloud9 IDE, use the terminal to clone the repository:

```
git clone https://github.com/aws-solutions-library-samples/guidance-for-simple-visual-search-on-aws
```

2. Change to the repository root folder:

```
cd guidance-for-simple-visual-search-on-aws
```

3. Initialize the Python virtual environment:

```
python3 -m venv .venv
```

4. Activate the virtual environment:

```
source .venv/bin/activate
```

5. Install the necessary python libraries in the virtual environment:

```
python -m pip install -r requirements.txt
```

6. Install the necessary node libraries in the virtual environment with your relevant package manager. For example with npm:

```
npm install
```

7. Verify that the CDK deployment correctly synthesizes the CloudFormation template:

```
cdk synth
```

8. Deploy the guidance:

```
cdk deploy
```

Deployment Validation

To verify a successful deployment of this guidance, open CloudFormation console, and verify that the status of the stack named `VisualSearchStack` is `CREATE_COMPLETE`.

Running the Guidance

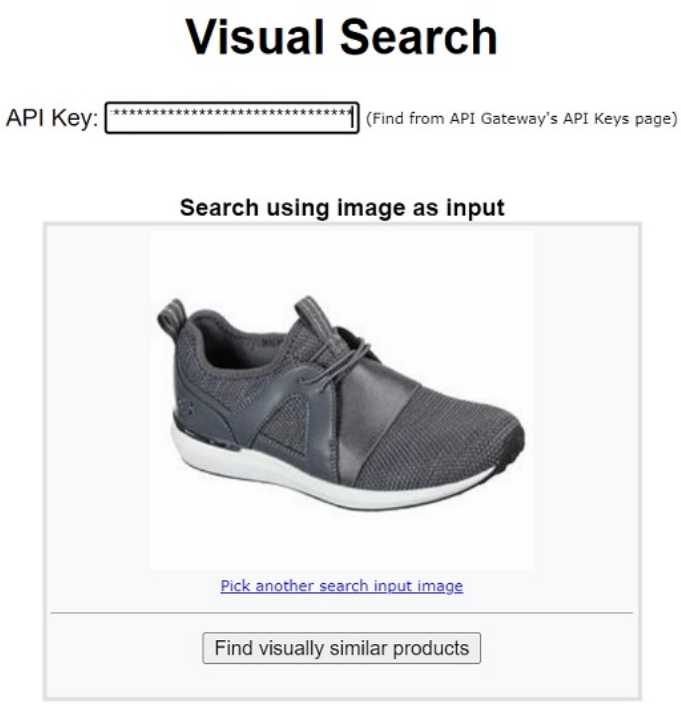
Ingest products into the OpenSearch vector database

- Open AWS Console and go to Lambda
- Select the checkbox next to function prefixed with `VisualSearchStack-VisualSearchProductIngestionLamb`
- Select "Actions"
- Select "Test"
- For Test event, select “Test” to run the lambda function
- This will ingest the product data into Amazon OpenSearch serverless by downloading the product.json from the S3 bucket and product images from Berkeley's S3 bucket s3://amazon-berkeley-objects. It also copies the product images to the local S3 bucket.

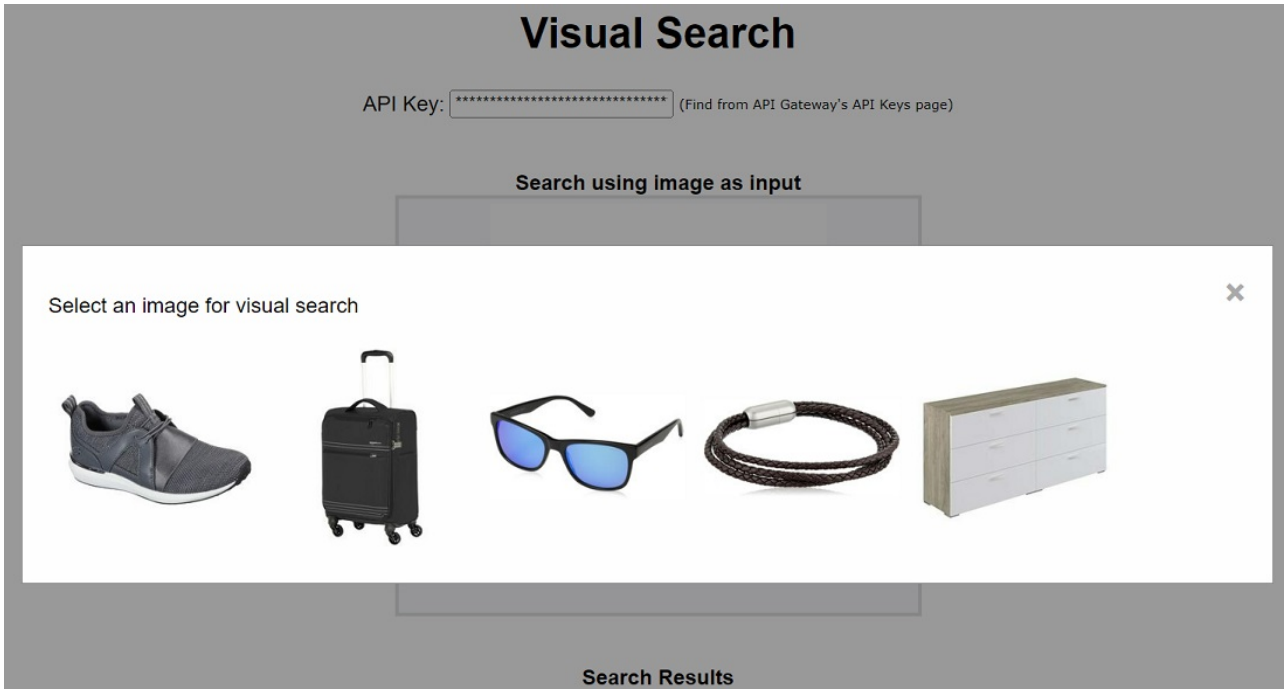
Do visual search

From UI

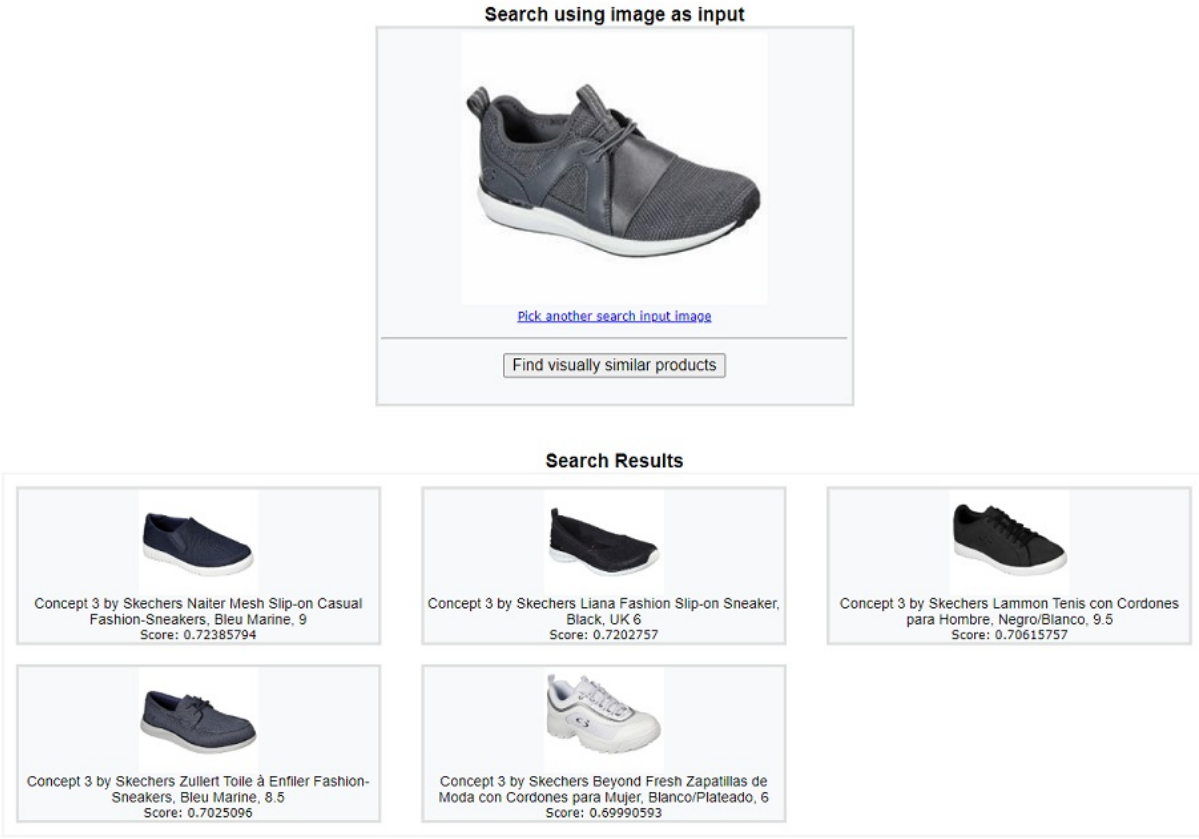
- Open API Gateway's `prod` stage URL which looks like `https://xxxxx.execute-api.<region>.amazonaws.com/prod`
  - You can get the API Gateway's URL from the `Outputs` of the CDK execution.
  - Alternatively, you can go to API Gateway in AWS Console, select VisualSearchAPIGateway API, select `Stages` from left navigation sidebar, select `prod` stage and copy the value of `Invoke URL`.



- This shows a sample UI that can be used for visual search.
- Select one of the given images as input.



- Provide the API Key in the API Key text box.
  - To get the API Key, go to API Gateway in AWS Console, select `API Keys` from the left navigation sidebar, find the API Key of Visual Search and copy its API Key.
- Click on "Find visually similar products". The search results would be shown.

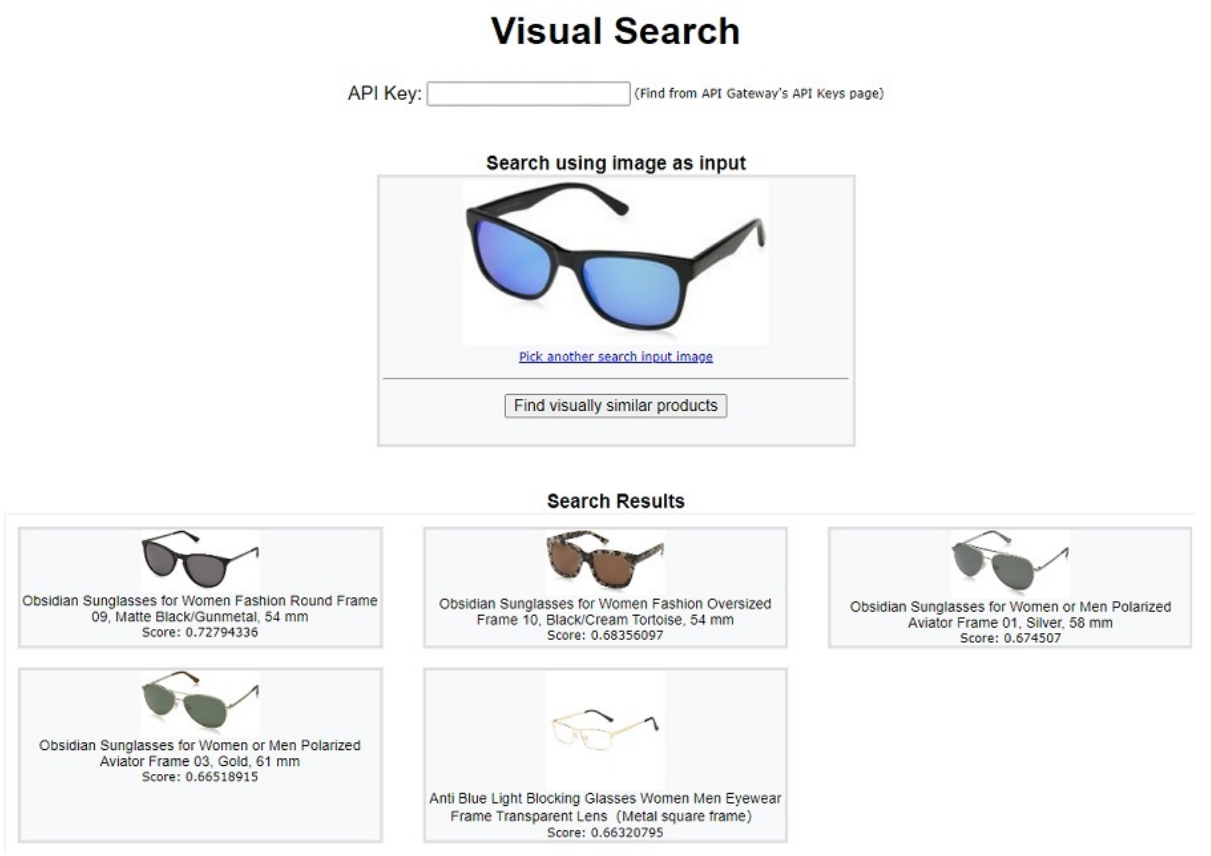


Through API

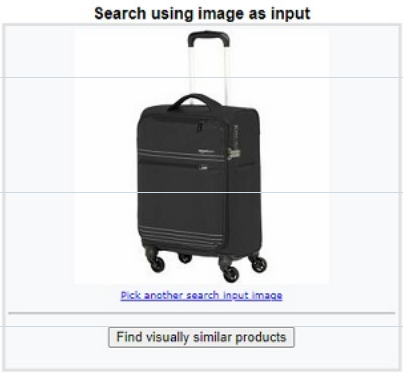
- Open AWS Console and go to API Gateway
- Go to the Visual Search API
- Find the `prod` stage URL and do `POST https://xxxxx.execute-api.<region>.amazonaws.com/prod/products/search` by passing a JSON in the format `{"content": "<base64 encoded image>"}`

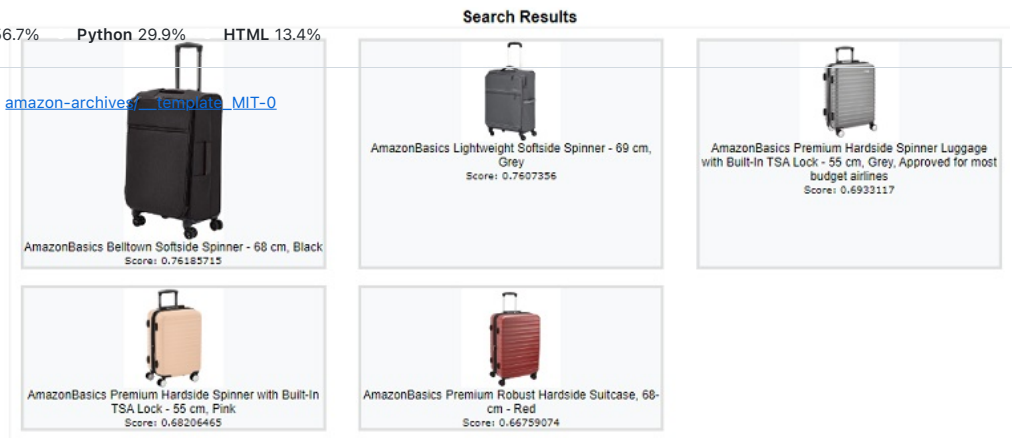
Sample searches

Sample search - Sunglasses



Sample search - Suitcases





Releases

No releases published

Packages

No packages published

Contributors 5

Languages

JavaScript 56.7% **Python** 29.9% **HTML** 13.4%

Generated from amazon-archives/__template_MIT-0

Next Steps

Several improvements can be made to make this code production ready.

- Use opensearch-py's bulk load capabilities for inserting data into the vector store for better performance.
- Use Amazon Bedrock's batch inference API during product ingestion.
- Load multiple images of a product.
- Filter the search results from OpenSearch to remove duplicates.
- Deploy the OpenSearch and Lambda in a VPC.
- Consider using Amazon Bedrock Provisioned Throughput pricing model if more capacity is needed.
- Move product ingestion code to ECS/EKS if the data set is large.

Cleanup

To delete the deployed resources, use the AWS CDK CLI to run the following steps:

1. Using the Cloud9 terminal window, change to the root of the cloned repository:

```
cd guidance-for-simple-visual-search-on-aws
```

2. Run the command to delete the CloudFormation stack:

```
cdk destroy
```

Authors

Rajesh Sripathi
Benedict Nartey-Tokoli
Dantis Stephen